Laboratório de Sistemas Operacionais – *sched_profiler*

Pedro Henrique Canali Jucewicz · PUCRS · novembro de 2026

1. Implementação

O programa recebe `<tamanho_buffer> <num_threads> <politica>`, aloca um buffer global e cria N threads (1 a 26), cada uma associada a uma letra (A, B, C...). Cada thread escreve sua letra na posição apontada pelo ponteiro global e o incrementa, até o buffer encher. O acesso ao buffer e ao ponteiro (inclusive o teste de fim do buffer) é protegido por um **semáforo binário** (`sem_t` iniciado em 1). A política é aplicada a cada thread com `pthread_setschedparam` (prioridade 1 para FIFO/RR e 0 para as demais); `SCHED_LOW_IDLE` é definida no programa com o valor 7, o mesmo usado no kernel modificado, pois a glibc não a conhece. Uma **barreira** (`pthread_barrier_t`, N+1 participantes) garante que nenhuma thread escreva antes de todas estarem criadas e com a política aplicada. Ao final, o programa imprime o buffer, o resumo (letras consecutivas iguais agrupadas) e quantas vezes cada thread foi escalonada (número de blocos de cada letra).

2. Ambiente e parâmetros

QEMU i386 com **1 núcleo** (`--smp cpus=1`), kernel Linux 4.13.9 com a política `SCHED_LOW_IDLE`, distribuição gerada com Buildroot. Parâmetros: **buffer de 5.000.000 posições e 4 threads** (≈ 2 s de execução no emulador, tempo suficiente para várias fatias de 100 ms do RR). Cada política foi executada uma vez com o trace capturado durante a execução:

```
trace-cmd record -o rr.dat -e sched:sched_switch \
    sh -c "sched_profiler 5000000 4 SCHED_RR > rr.txt"
```

3. Saída de exemplo (SCHED_RR)

```
$ sched_profiler 5000000 4 SCHED_RR
AAAAAAAAAAAAAAAA... (481.916 A) ...BBBBBBBB... (634.758 B) ...CCCC... (111.065 C) ...

Resumo:
ABCDABCDABCDAB
A = 4
B = 4
C = 3
D = 3
```

4. Resultados

Política	Resumo (início)	Escalonamentos A / B / C / D	Troca entre threads
`SCHED_LOW_IDLE`	`BCDABCDABCADBCADCBCA...`	100 / 95 / 101 / 92	~500 trocas, a cada ~4 ms, sem ordem fixa
`SCHED_IDLE`	`BCDABDCABCDBACDABDCA...`	119 / 121 / 125 / 124	~670 trocas, a cada ~4 ms, sem ordem fixa
`SCHED_RR`	`ABCDABCDABCDAB`	4 / 4 / 3 / 3	a cada ~100 ms, ordem circular fixa
`SCHED_FIFO`	`A`	1 / 0 / 0 / 0	nenhuma: A executa até terminar

5. Análise

SCHED_FIFO. A primeira thread (A) recebe a CPU na liberação da barreira e não a cede às demais de mesma prioridade, pois a FIFO não possui fatia de tempo: ela escreve o buffer inteiro sozinha (resumo `A`). O trace mostra uma única interrupção de ~50 ms após ~950 ms de execução, causada pelo **RT throttling** do Linux (`sched_rt_runtime_us` = 950 ms por período de 1 s): durante a pausa a CPU ficou ociosa, e A retomou a execução na frente da fila. Quando A termina, B, C e D executam por microssegundos, encontram o buffer cheio e terminam (estado `x` no trace).

SCHED_RR. As threads se revezam em ordem circular fixa (A, B, C, D) em fatias de ~100 ms (trocas com estado `R`, isto é, por preempção). Os blocos não têm todos o mesmo tamanho porque, quando uma thread é preemptada segurando o semáforo, as demais acordam, encontram-no ocupado e voltam a dormir (estado `S`), e a CPU retorna à thread que o detém. O RT throttling também atua, já que as quatro threads somadas ocupam a CPU continuamente.

SCHED_IDLE e SCHED_LOW_IDLE. Ambas são escalonadas pelo CFS, que escolhe sempre a thread com menor tempo de CPU acumulado (*vruntime*). Por isso as trocas ocorrem a cada ~4 ms (tick de 250 Hz), sem ordem fixa, e a divisão é equilibrada (≈ 25% para cada thread). Como estas threads têm prioridade inferior à do `main` (normal), nenhuma executa durante a criação: o `main` chega primeiro à barreira e a última thread a chegar libera a largada. Os padrões das duas políticas são semelhantes porque todas as threads usam a mesma política: a `SCHED_LOW_IDLE` (peso menor e preemptada por qualquer outra política, inclusive `SCHED_IDLE`) só se diferencia ao competir com tarefas de outras políticas.

Conclusão. As políticas de tempo real (FIFO, RR) passam à frente de qualquer tarefa normal e produzem poucos blocos longos, diferenciando-se pela existência de fatia de tempo; as políticas do CFS (IDLE, LOW_IDLE) produzem centenas de blocos curtos e uma divisão justa da CPU.

6. Visualização no KernelShark (`sched:sched_switch`). Em cada linha de thread: verde = executando, retângulo vermelho = pronta (na fila), linha preta = dormindo.

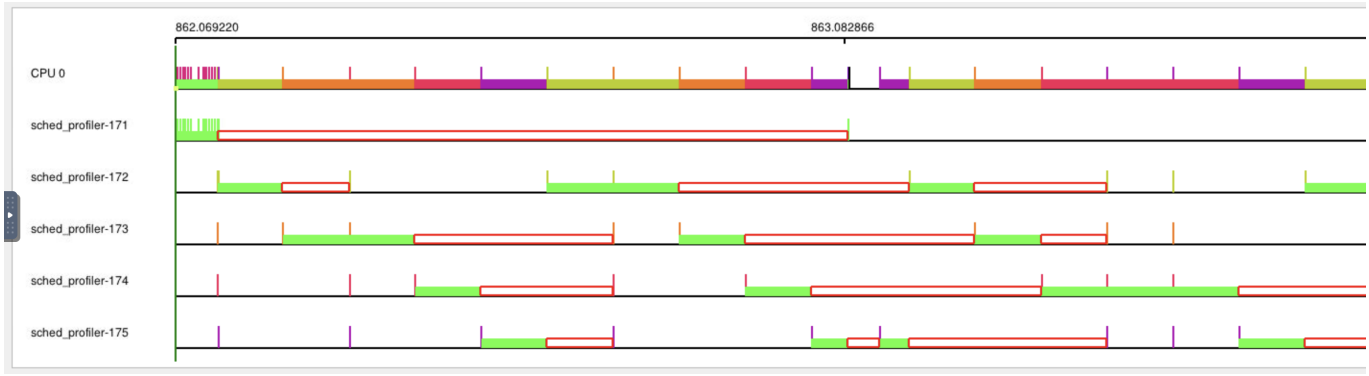


Figura 1 – SCHED_RR (main = 171, A–D = 172–175): fatias de ~100 ms em ordem circular fixa.



Figura 2 – SCHED_FIFO (main = 186, A–D = 187–190): A executa sozinha; o marcador indica a pausa de ~50 ms do RT throttling; B, C e D só executam no fim, e terminam.

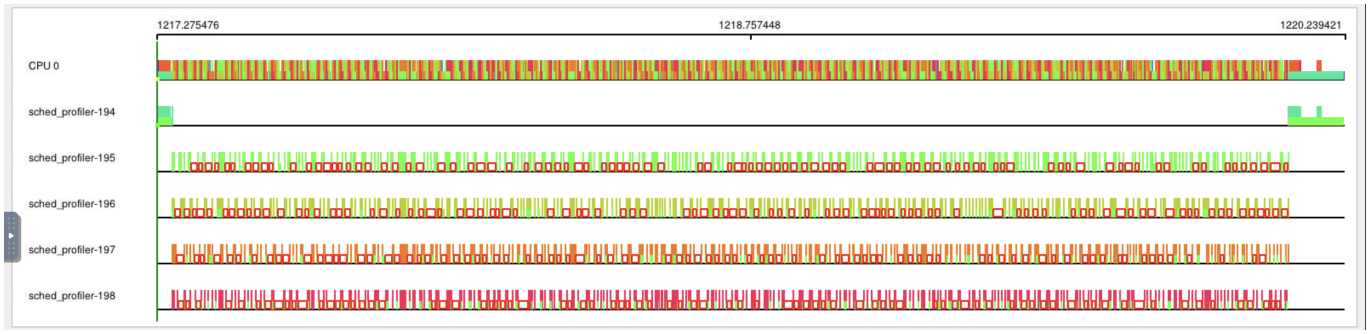


Figura 3 – SCHED_IDLE (main = 194, A–D = 195–198): trocas a cada ~4 ms, sem ordem fixa; o main só executa antes e depois.

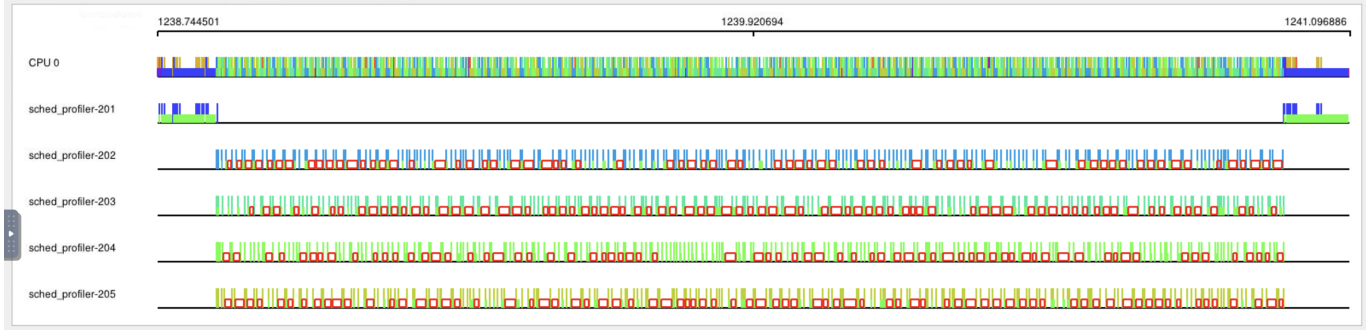


Figura 4 – SCHED_LOW_IDLE (main = 201, A–D = 202–205): padrão semelhante ao SCHED_IDLE.